# QC App

Loudness compliance for broadcast, streaming and podcast delivery

# User Manual and Technical Documentation

2026-08-03 · revision fd48029

7,830 lines of source · 2,424 lines of tests · 94 automated tests

*Measurement conforms to ITU-R BS.1770-4 and EBU Tech 3341/3342. The delivery specifications shipped with the application are seed values and must be verified against each platform's current published specification before a verdict is relied upon.*

# Contents

# Part I

User Manual

## 1. What this application is

QC App measures audio files against broadcast, streaming and podcast delivery specifications and tells you, per platform, whether the file passes. It is a measurement and reporting tool: it reads files, it never writes audio, and it never alters the material you point it at.

Every measurement follows ITU-R BS.1770-4 and EBU Tech 3341/3342: integrated, short-term and momentary loudness, loudness range, and true peak by oversampling. Alongside those it runs sample-domain checks that a loudness figure alone will never catch — clipping, polarity problems, and mono compatibility.

**What it deliberately does not do:**

- Correct or render audio. It measures and advises; fixing the file is your job, in your own tools.
- Measure a live input. An output device is opened only so you can audition a file.
- Handle surround or immersive audio. Mono and stereo only.
- Open video containers. Audio files only.

## 2. Installing and running

The application is a single executable with no installer and no runtime dependencies beyond Windows 10 or later. Copy it wherever you like and run it. On first launch it writes an editable copy of its delivery specifications to your application data folder (see section 12).

You can also hand it files or a folder on the command line, which is what a shell association does:

```
"QC App.exe" "D:\Deliveries\episode_01.wav"
"QC App.exe" "D:\Deliveries\Series 3"
```

A single file opens in detail view. Several files, or a folder, start a batch. This is exactly the routing a drag-and-drop takes.

## 3. The interface


*Figure 1 - detail view after analysing a single file.*

The window has six regions:

| Region | What it is for |
|---|---|
| **Header bar** | Opening files, loading a dialogue stem, exporting, and the subfolder toggle. The right-hand text shows the source format and any warning about the target definitions. |
| **Targets column** | Every delivery specification loaded from targets.json. Tick the ones you are delivering to. The selection is remembered between launches. |
| **Measurement strip** | The headline numbers for the file on screen. |
| **Verdict list** | One block per selected target: a status, the checks behind it, and a fix hint when something failed. |
| **Transport** | Play, stop, and the playback position. |
| **Loudness graph** | Loudness over time, the target band, true-peak overs, and the playhead. |

# 4. Analysing a file

Drop an audio file anywhere in the window, or use **Open file...**. Analysis runs on a background thread and faster than real time; the window stays responsive throughout. Nothing is written anywhere unless you explicitly export.

If a file cannot be decoded, the status line says so and names the extension and the formats this build supports. Analysis never fails silently.

# 5. Reading the measurements

| Figure | Meaning |
|---|---|
| **Integrated** | Loudness of the whole programme, gated per BS.1770-4. This is the number every delivery specification is written against. |
| **Short-term max** | Loudest 3-second window. |
| **Momentary max** | Loudest 400-millisecond window. |
| **LRA** | Loudness range: the spread between the quiet and loud parts of the programme, in LU. A large value means wide dynamics, which some platforms discourage. |
| **True peak** | The highest level the waveform reaches between samples, found by oversampling. It can exceed sample peak by a decibel or more, which is why a file that looks safe in a DAW can still clip a listener's decoder. |
| **Sample peak** | The highest individual sample value, in dBFS. |
| **PLR** | Peak-to-loudness ratio: true peak minus integrated loudness. A rough measure of how much dynamic range is left. |
| **Duration** | Length of the file. |

A value shown as **--** was not measured, rather than measured as zero. A file that is silent, or shorter than one 400 ms gating block, has no integrated loudness at all, and the application will not invent one.

# 6. Reading the graph

The graph plots the programme against the first target you selected.

- The **white trace** is short-term loudness (3-second window).
- The **teal trace** behind it is momentary loudness (400 ms), which moves faster and shows transients.
- The **green band** is the selected target's tolerance, and the line through it is the target itself.
- The **dashed white line** is the file's integrated loudness — where the whole programme sits.
- **Red columns** mark true-peak overs, drawn the full height of the plot so they can be found against the trace.
- The **blue playhead** follows playback. Click or drag anywhere in the plot to seek there.

The vertical scale follows the material rather than a fixed window, so a quiet podcast and a loud master are both legible.

# 7. Targets and verdicts

Each selected target produces one of four results:

| Status | Meaning |
|---|---|
| **PASS** | Loudness is within tolerance and true peak is under the ceiling. |
| **WARN** | Deliverable, but something is worth looking at: clipping, out-of-phase content, poor mono compatibility, or a loudness range wider than the target's guidance. |
| **FAIL** | Loudness is outside tolerance, or true peak exceeds the ceiling. |

| Status | Meaning |
|---|---|
| **NOT MEASURED** | The measurement this target needs was not available — a silent file, or a dialogue-gated target with no stem loaded. It is never rendered as a pass. |

> **Loudness and true peak are pass-or-fail only**
>
> Warnings are reserved for the sample-domain checks and loudness-range guidance. A green result therefore always means *this is deliverable*, never *this is deliverable apart from the bit that is not*. In particular, a file whose loudness is within tolerance but whose true peak is over the ceiling is a **FAIL**.

Some platforms normalise on playback rather than rejecting a delivery. For those, loudness is reported with the offset from target but is not a failure; only the true-peak ceiling can fail.

# 8. Fix hints

When a target fails, the application states the change that would fix it. The gain figure and the resulting true peak are exact, because gain is linear. Only the amount of limiting is an estimate, and it is worded as one.

| Situation | What you are told |
|---|---|
| Loudness off, headroom available | *Apply -2.3 dB gain -> -14.0 LUFS, true peak lands at -1.4 dBTP. Passes.* |
| Loudness off, gain would breach the ceiling | *Apply +1.8 dB gain -> -14.0 LUFS, but true peak would hit -0.2 dBTP (ceiling -1.0). Needs limiting, around 0.8 dB of gain reduction.* |
| Loudness fine, true peak over | *Loudness passes. Reduce true peak by 1.3 dB - limiter ceiling at -1.0 dBTP.* |
| Loudness range wide | *LRA 21.4 LU is wide for this target - compression or level-riding needed.* |

# 9. Dialogue-gated targets

Some specifications — Netflix among them — are written against dialogue-gated loudness, measured only over the intervals where dialogue is present. That is a different measurement, not a different threshold, so it cannot be answered from the programme figure.

Load a dialogue stem with **Dialogue stem...**. The stem must be the same sample rate and the same length as the mix; anything else is rejected rather than silently stretched to fit. The application then measures the mix over the intervals where the stem is active.

> **This is not Dolby Dialog Intelligence**
>
> The result will not agree with a proprietary dialogue detector sample for sample. It is an honest measurement of the mix over the dialogue intervals your stem defines. Without a stem, dialogue-gated targets report NOT MEASURED rather than guessing.

# 10. Batch mode

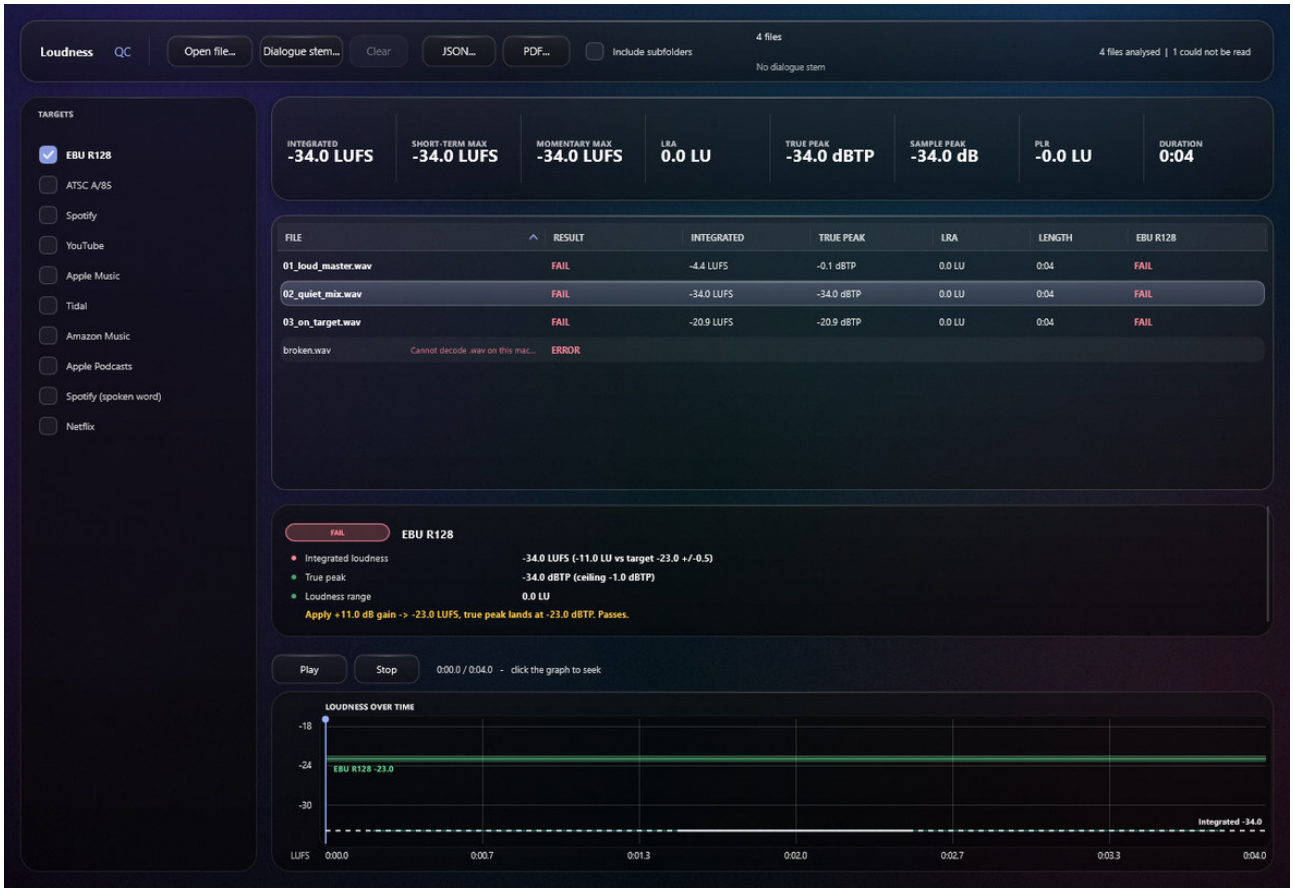


*Figure 2 - batch view. The failed file carries its reason on the row.*

Drop a folder, or several files, to start a batch. Files are analysed in parallel and the table fills in as they finish. Click any row to see that file in the measurement strip, the verdict list and the graph.

- Every column sorts, including the per-target status columns.
- A file that cannot be read becomes an **ERROR** row carrying its reason, and the run continues. One bad file in a delivery folder never costs you the other ninety.
- Non-audio files are skipped silently rather than filling the table with errors.
- **Include subfolders** is off by default, so dropping a project folder does not pull in every bounce and stem underneath it.
- Changing which targets are selected re-judges every row instantly — no file is read again.

# 11. Playback

The file on screen can be auditioned while its report is visible. **Play** and **Stop** control the transport; the readout shows position and length. Click or drag anywhere in the graph to seek, which is usually faster than scrubbing, because the graph is where you can see the passage you want to hear.

The output device is opened the first time you play something, not at launch. A machine with no output device analyses normally; the transport is disabled and says why. Playback never influences a measurement.

# 12. Delivery specifications

Targets are data, not code. On first run the application writes a copy of its defaults to:

```
%APPDATA%\QC App\targets.json
```

That copy is preferred from then on, so an application update never overwrites a correction you made. Edit it and restart. Each entry looks like this:

```
{
  "id": "ebu-r128",
  "name": "EBU R128",
  "integratedLufs": -23.0,
  "toleranceLu": 0.5,
  "maxTruePeakDb": -1.0,
  "gating": "program",
  "lastVerified": "2026-08-03"
}
```

| Field | Required | Meaning |
|---|---|---|
| `id` | yes | Stable identifier used in exports. Do not change it once reports exist. |
| `name` | yes | What appears in the interface. |
| `integratedLufs` | yes | Target integrated loudness. |
| `toleranceLu` | no | Permitted deviation. Omit for platforms that normalise on playback rather than reject a delivery. |
| `maxTruePeakDb` | yes | True-peak ceiling in dBTP. |
| `gating` | no | Either "program" (the default) or "dialogue". |
| `maxLoudnessRangeLu` | no | Guidance only. Exceeding it warns, never fails. |
| `lastVerified` | no | ISO date. Empty means the numbers have never been checked. |

> **Verify the numbers before you rely on a verdict**
>
> The shipped values are seeds, and platform specifications drift. Every target whose lastVerified is empty is flagged in the interface and printed on every PDF page as *spec not verified*. A verdict against a specification nobody has checked is worth less than no verdict at all, so the application says so rather than looking authoritative.

An entry that cannot be understood is skipped and reported, never silently dropped — a target missing from the list would otherwise look like a pass.

# 13. Exporting

**JSON** writes a complete machine-readable result for the file on screen: source metadata, every measurement, the sample-domain checks, each verdict with the specification it was judged against, true-peak overs with timestamps, and the full loudness time series.

**PDF** writes a print-ready QC report, one page per file. In batch mode it covers every file analysed so far, including those that failed to read — a report that silently omitted them would overstate what was checked.

> **Values that were never measured are written as null**
>
> JSON has no representation for infinity. A silent file's loudness is written as null, never as a number, because a consumer reading -inf as a level would draw exactly the wrong conclusion. Status tokens in JSON (pass, warn, fail, notMeasured) are deliberately separate from the labels shown on screen, so renaming something in the interface cannot break a tool parsing your output.

# 14. Supported formats

| Family | Extensions | How it is decoded |
|---|---|---|
| Uncompressed | `.wav .bwf .aiff .aif` | Native. All bit depths and sample rates. |
| Lossless / open | `.flac .ogg` | Native. |
| MP3 | `.mp3` | Native decoder; no system codec needed. |
| AAC family | `.m4a .aac .adts .m4b` | Windows Media Foundation, using codecs already on the system. |
| Windows Media | `.wma .wmv .asf .wm` | Windows Media Foundation. |

Video containers are out of scope. Files with more than two channels are rejected, with their channel count stated.

# 15. Troubleshooting

| What you see | What it means and what to do |
|---|---|
| *Cannot decode .xyz on this machine* | The format is not supported by this build; the message lists what is. Convert the file, or check that the extension matches the actual contents. |
| *This build measures mono and stereo only* | The file has more than two channels. Surround is out of scope; deliver a stereo fold-down or measure the stems separately. |
| *File is silent or falls below the -70 LUFS gate* | No block in the file survived gating. Either the file really is silent, or it is shorter than one 400 ms block. |
| *Dialogue stem required* | A dialogue-gated target is selected but no stem is loaded. Load one, or deselect the target. |
| *Dialogue stem is N s but the mix is M s* | They must be the same length and sample rate. Export the stem again from the same session. |
| *No audio output available* | Playback could not open a device. Analysis is unaffected — the report on screen is valid. |
| *N target(s) not yet verified* | A reminder that targets.json still carries unchecked numbers. See section 12. |
| *ERROR* row in a batch | That file could not be read; the reason is on the row. The rest of the run is unaffected. |

# Part II

Technical Documentation

## 16. Architecture

The codebase is four layers, and the dependency rule runs one way only: nothing in a lower layer knows the layer above it exists.

| Layer | Directory | Depends on | What lives there |
|---|---|---|---|
| Engine | Source/Engine | nothing but the standard library | K-weighting, loudness metering, true peak, sample-domain checks, the result struct. |
| Verdict | Source/Verdict | Engine | Delivery specifications, pass/fail rules, fix hints. |
| Report | Source/Report | Engine, Verdict | JSON serialisation, the PDF primitive writer, and the report layout. |
| App | Source/App | everything below, plus JUCE | File reading, batching, playback, the interface. |

> **The first three layers contain no JUCE**
>
> Measurement, judgement and reporting are plain C++17 with no framework dependency at all. That is not purism: it means the entire analysis path builds and runs in a test binary with no window, no audio device, and no network fetch, which is what makes the suite fast enough to run on every change.

The consequence worth internalising is that **AnalysisResult** is the seam. It is plain data. The graph, the verdict panel, the JSON writer and the PDF report all consume it and nothing else, so a result can be constructed by hand in a test without a file existing anywhere.

## 17. Building

CMake with CPM, which fetches a pinned JUCE on first configure. A clean clone needs nothing installed beyond a compiler and CMake.

```
cmake -S . -B build -DQC_BUILD_APP=ON
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure
```

| Option | Default | Effect |
|---|---|---|
| `QC_BUILD_TESTS` | ON | Builds the test binaries. |
| `QC_BUILD_APP` | OFF | Builds the application. Off by default so a clean clone configures and tests with no network access; turning it on downloads JUCE. |
| `QC_BUILD_UI_SNAPSHOT` | OFF | Builds qc_uishot, the offscreen interface renderer. |
| `QC_BUILD_ICON_TOOL` | OFF | Builds qc_makeicon, the icon generator. |

JUCE is pinned to 8.0.9. JUCE 9 exists but nothing here has been verified against its API changes; bump the tag in `Source/App/CMakeLists.txt` and run the application before trusting it. `JUCE_DISPLAY_SPLASH_SCREEN` is left at its default, because turning it off is a licence decision rather than a build decision.

# 18. The measurement engine

### *K-weighting*

BS.1770-4 publishes filter coefficients for 48 kHz only. Reusing those values at another sample rate shifts the corner frequency and biases every measurement taken at that rate, so **KWeighting.cpp** re-fits the analogue prototype and bilinear-transforms it at whatever rate is passed in. At 48 kHz this reproduces the published table to within 1e-9, which is asserted by a test — if that assertion ever fails, every measurement at every sample rate is wrong.

The filters run in double precision throughout. The RLB high-pass sits at 38 Hz, and in single precision its state decays into the noise floor over a long file.

### *Loudness metering*

**LoudnessMeter** consumes audio in arbitrary-sized chunks and reduces it immediately to one mean-square value per channel per 100 ms sub-block. Every published window is then an average over a whole number of those sub-blocks:

- Momentary is 4 sub-blocks (400 ms), hopped 1.
- Short-term is 30 sub-blocks (3 s), hopped 1.
- The gating blocks for integrated loudness are the 400 ms windows at 75% overlap — which is exactly the momentary series, so it is computed once.
- LRA windows are 30 sub-blocks hopped 10, per EBU Tech 3342.

The consequence is that memory grows with duration rather than with sample count: an hour of stereo costs well under a megabyte, so a feature-length file is no harder than a thirty-second spot. A trailing partial sub-block is deliberately discarded, because BS.1770-4 gates over complete blocks and including a short final one would bias the result upward.

Gating is exposed as free functions — `gatedBlockIndices` and `gatedLoudness` — rather than being buried in the class, because dialogue gating needs to run the same arithmetic over a restricted set of blocks.

### *True peak*

**TruePeakDetector** oversamples with a Kaiser-windowed sinc polyphase interpolator: 4x below 96 kHz, 2x below 192 kHz, none above, where inter-sample peaks are already resolved. The filter is generated rather than taken from the table in BS.1770-4 Annex 2, because that table is specified for 48 kHz 4x only and generating it keeps the same code correct at other rates. Each polyphase branch is normalised to unit DC gain; without that the branches differ by a fraction of a decibel and a steady tone appears to wobble.

Alongside the overall maximum the detector keeps a per-channel envelope of the highest interpolated magnitude in each 10 ms slice. Over events for any ceiling are derived from that envelope, which is why changing the selected target re-reports overs instantly without re-reading the file, and why memory is bounded by duration rather than by how loud the material is.

> **An earlier design stored every over candidate individually**
>
> It would have hit its cap after about four seconds of any loud master, and its event merging failed across interleaved channels. The envelope replaced it before the first build. Bounded-by-duration is the property that matters here.

### *Sample-domain checks*

**QualityAnalyser** covers what loudness cannot: sample peak per channel, runs of consecutive samples at or above the clip threshold, and stereo correlation. Correlation is evaluated in 100 ms blocks so the reported figure means *this much of the programme was out of phase* rather than *the whole file averaged out to roughly zero*, which is what a single global figure gives you on material that swings between wide and mono. Blocks below a silence floor are excluded, or every fade would be flagged.

### *Orchestration and mono compatibility*

**AudioAnalyser** owns the meters and feeds them one pass of the audio. For stereo it also feeds a second loudness meter with the mono downmix, computed as (L + R) / sqrt(2) rather than (L + R) / 2. That normalisation is power-preserving: identical channels sum to the same loudness as the pair and report zero loss, while fully decorrelated content lands at 3 dB. With a /2 downmix every mono-compatible file would report a 3 dB loss and the check would be useless.

A stereo file whose mono sum falls below the gate has cancelled itself out entirely. That reports as infinite loss, and the verdict renders it in words rather than as a number. An earlier version reported it as *zero* loss, because the comparison was skipped when the mono figure was unmeasurable — the worst possible result presented as the best.

### *Dialogue gating*

`computeDialogueGatedLoudness` takes the programme's gating-block powers and the stem's, gates the stem normally to find where dialogue is, and integrates the programme over those blocks.

There is one subtlety that a test caught. Gating blocks are 400 ms hopped 100 ms, so the three blocks straddling the start or end of a dialogue passage are only partly dialogue. The stem passes them — a quarter of a phrase is far above the gate — but in the programme those same blocks contain whatever else is playing. On a mix where dialogue gives way to loud music that pulled the reading 1.3 dB high. The active set is therefore eroded by one window length at each edge, leaving only blocks that are dialogue the whole way through. The cost is that passages shorter than about 0.7 s drop out, which is the right way to be wrong: an under-sampled dialogue figure beats one contaminated by music.

# 19. The verdict engine

`evaluate(result, target)` is pure: it reads an AnalysisResult and a Target and returns a TargetVerdict. No I/O, no state, no framework.

The status model has four values and one ordering rule:

```
pass  <  warn  <  notMeasured  <  fail
```

A verdict takes the worst status among its checks, and a batch row takes the worst across its verdicts. **notMeasured** outranks **warn** deliberately: an unjudgeable target must not be hidden behind a warning about something else.

An empty verdict list returns notMeasured, not pass. This mattered in batch mode, where a row with no targets selected would otherwise have rendered green — nothing judged is not the same as nothing wrong.

Each verdict carries the specification it was judged against, including its lastVerified date, so a report can state the numbers and their provenance rather than making a reader look them up.

# 20. Reporting

### *JSON*

**JsonReport** is hand-written rather than built on juce::var, which keeps it in the dependency-free layer and lets the engine test binary cover it. The rules that matter:

- Anything non-finite is written as `null`. JSON cannot represent infinity, and a consumer reading -inf as a level would draw exactly the wrong conclusion.
- Status tokens are separate from the interface labels, so renaming a label cannot break a downstream parser.
- True-peak overs are listed per target with timestamps. The list is capped, but the count is always the real total — a capped list must never understate how bad a file is.
- Backslashes and quotes are escaped, which matters because every path on this platform is full of backslashes.

### *PDF*

JUCE provides no PDF or printing API on Windows. Rather than ship a library for one file format, **PdfWriter** emits PDF operators directly: filled and stroked rectangles, polylines, and text in the standard Helvetica faces. Using the standard fonts means nothing is embedded, so a report stays small and opens anywhere, and the whole path stays in the dependency-free layer.

Text widths are approximated deliberately. Exact widths would mean shipping Helvetica metrics for three faces, and nothing in the report is centred or right-aligned against a measured width; the approximation is used only to decide where to truncate a long path.

**PdfReport** lays out one A4 page per file. The loudness series is decimated to 900 points, keeping the loudest value in each bucket rather than the mean, so peaks survive. An hour of audio drawn point for point would be megabytes of detail no printer resolves; a test pins an hour-long file under 200 kB.

> **The PDF tests walk the cross-reference table**
>
> Every byte offset is checked to resolve to the object it claims. A wrong offset there opens as a blank page, and no substring assertion would notice. Escaping is covered too: an unescaped parenthesis terminates a PDF string early and corrupts the rest of the page.

# 21. The application layer

### *File reading*

**FileAnalysisJob** streams a file in 32,768-sample blocks into an AudioAnalyser, polling a cancellation callback and reporting progress between blocks. Failures name what went wrong: the extension that could not be decoded and what this build supports, the channel count that is out of scope, or the specific mismatch between a mix and its dialogue stem. A stem of a different rate or length is rejected rather than truncated, because gating against it would produce a confident number about the wrong intervals.

### *AAC and M4A*

JUCE's WindowsMediaAudioFormat registers only .wmv/.asf/.wm/.wma, which leaves out the container podcast and streaming deliverables actually arrive in. **MediaFoundationAudioFormat** fills that gap with an IMFSourceReader-backed juce::AudioFormat, using decoders already present on every supported Windows install.

- Float output is requested, with 16-bit PCM as a fallback converted on the way in, so the rest of the application never sees which path was taken.
- Position is tracked from the decoder's own timestamps, so a seek lands where it claims.
- Any shortfall is zero-filled: a compressed file's duration is metadata, and encoder padding routinely overstates what actually decodes.
- Media Foundation decodes from a path, so the format only accepts a juce::FileInputStream — and fails cleanly rather than silently for anything else.

The tests do not commit an encoded fixture. They encode AAC with the system encoder via IMFSinkWriter and decode it back, because a committed binary proves nothing about the machine running the suite.

### *Batching*

**BatchAnalyser** runs files through a thread pool capped at four workers rather than one per core. Analysis is memory-bandwidth bound rather than compute bound, so saturating every core buys little and makes the machine unusable while a folder is checked.

Entries are mutated only on the message thread: a worker produces an outcome and hands it over with `MessageManager::callAsync`. That is why there is no lock in the class. Callbacks from a superseded run are dropped, since a run can be replaced while its callbacks are still in flight.

`reevaluate()` re-judges completed entries against a new target selection without touching the disk — the measurements do not change, only what they are compared with.

### *Playback*

**PlaybackEngine** wraps an AudioDeviceManager, an AudioSourcePlayer and an AudioTransportSource. The device is opened lazily on first use, output-only, so a session spent checking files never takes a device from anything else and never prompts for microphone permission.

> **The read-ahead thread initialises COM for itself**
>
> The Media Foundation reader creates its objects on the thread that opened the file and then has samples pulled from the read-ahead thread. Calling into those objects from a thread with no apartment fails at run time — and only for compressed formats, so WAVs would play fine and podcasts would fail in the field. This is the kind of defect that ships if the threading is not thought about explicitly.

### *Target loading*

**TargetLibrary** parses targets.json, preferring a user copy in the application data directory over the compiled-in default and writing that copy on first run. Malformed entries are skipped and reported rather than dropped, and targets with no lastVerified date are collected so the interface can say how many are unverified.

## 22. The interface layer

**GlassStyle** holds the entire visual language: palette, panel metrics, and three depths (recessed, raised, floating) that everything is drawn at. Panels darken what is behind them rather than lightening it. Tinting glass

white looks convincing over a photograph, but this interface is almost entirely light text, and a lightened panel costs most of its contrast. **GlassLookAndFeel** restyles the stock JUCE controls; leaving them is the single biggest giveaway in a themed JUCE application.

The backdrop is drawn at a tenth scale and resampled up rather than blurred at full size — cheaper, and smoother falloff than a large-radius blur. A faint grain is laid over it, without which gradients that wide band visibly on 8-bit displays.

**MainComponent** owns the layout and the wiring. The verdict list takes the height its content needs and the graph takes the rest, rather than the graph taking a fixed slice and leaving a dead band across the middle whenever one or two targets are selected.

**LoudnessGraph** plots the series, the target band, the overs and the playhead, and is also the scrub surface: it is where you can see the passage you want to hear. The playhead repaints only the two columns it moved between, because at thirty updates a second across a full-width plot, repainting the whole graph is visible as a stutter.

# 23. Threading model

| Thread | What runs on it | Rules |
|---|---|---|
| Message thread | All interface work; every mutation of batch entries; all callbacks delivered from workers. | Never block it. Analysis and playback both hand results back to it. |
| Analysis pool | Single-file analysis (one job). | Produces an outcome, posts it with callAsync. Polls a cancellation flag between blocks. |
| Batch pool | Up to four concurrent file analyses. | Same contract. Never touches shared state directly. |
| Audio device | Pulls from AudioTransportSource. | No allocation, no locks, no file reads. Buffering happens on the read-ahead thread. |
| Read-ahead | Fills the transport's buffer from the file. | Initialises COM for itself so Media Foundation readers work from here. |

# 24. Testing

There are 94 tests across two binaries, run by ctest. The split is deliberate.

| Binary | Covers | Dependencies |
|---|---|---|
| `qc_engine_tests` | K-weighting, loudness and gating, LRA, true peak, sample-domain checks, dialogue gating, the verdict engine, JSON, PDF. | None at all. Builds and runs with no JUCE. |
| `qc_file_tests` | File reading, AAC round trips, batching, playback. | JUCE audio modules; only built when the application is. |

The harness is a hand-rolled 100-line header rather than a package, so the engine suite builds from a clean clone with no network access and no third-party dependency to justify. If it outgrows that, Catch2 is the obvious replacement.

What is worth understanding is *how* the measurements are validated:

- Loudness tests do not compare against recorded fixtures. They predict what the meter should read from the filter's transfer function and the BS.1770-4 summation, then assert the meter agrees within 0.05 dB. That validates the implementation against theory rather than against a previous run of itself.
- The 48 kHz coefficient table is asserted directly, because if the derivation drifts every measurement at every rate is wrong.
- Gating, linearity, channel summation and chunk-size independence each have a test.
- The true-peak test uses a sine at fs/4 offset by a quarter cycle: every sample lands at -3 dBFS while the waveform reaches full scale between samples. Sample peak says -3, true peak says 0. That is the case the detector exists for.
- AAC fixtures are encoded by the test at run time and decoded back.
- Playback asserts the position actually advances on a real device, and skips with a printed note where no output device exists — a silently skipped test is indistinguishable from a passing one.

# 25. Developer tools

**qc_uishot** renders the real MainComponent to a PNG without opening a window. A screenshot of a running application cannot be taken repeatably or compared across a change; this can, and every layout defect fixed during the interface work was found with it rather than guessed at.

```
cmake -S . -B build -DQC_BUILD_APP=ON -DQC_BUILD_UI_SNAPSHOT=ON
cmake --build build --config Release --target qc_uishot
qc_uishot out.png 1280 880 "D:\Deliveries\episode_01.wav"
```

**qc_makeicon** draws the application icon and emits a preview strip of every size from 16 to 256 on light and dark backgrounds. The icon is generated rather than committed as an opaque bitmap so it can be adjusted without a graphics editor. The build consumes the committed PNG, not the tool.

# 26. Extending the application

### *Adding a delivery specification*

Edit targets.json. No code change, no rebuild. Set lastVerified once you have checked the numbers against the published specification.

### *Adding an input format*

Register another juce::AudioFormat in `getFormatManager()` in FileAnalysisJob.cpp. Analysis, playback and the file chooser all read from that one manager, so a format added there appears everywhere at once. Add its extensions to the test that asserts the promised formats are actually openable.

### *Adding a measurement*

Add the field to AnalysisResult, populate it in AudioAnalyser, and it becomes available to the verdict engine, the JSON writer, the PDF report and the interface at once. If it should affect a verdict, add a CheckResult in VerdictEngine and decide deliberately whether it warns or fails — the rule is that loudness and true peak fail, everything else warns.

### *Supporting surround*

The channel-count guards in LoudnessMeter and FileAnalysisJob are the gate. BS.1770-4 channel weighting (Ls/Rs at +1.5 dB, LFE excluded) goes in `LoudnessMeter::windowedPower`, which currently assumes all

weights are 1.0 and says so in a comment. The mono-compatibility check and the downmix would need rethinking.

# 27. File reference

| File | Lines | Purpose |
|---|---|---|
| Source/App/BatchAnalyser.cpp | 160 | Worker pool, message-thread state |
| Source/App/BatchAnalyser.h | 97 | Batch run interface |
| Source/App/BatchTable.cpp | 389 | Sortable rows, dynamic target columns |
| Source/App/BatchTable.h | 75 | Batch table interface |
| Source/App/FileAnalysisJob.cpp | 281 | Block streaming, stem validation |
| Source/App/FileAnalysisJob.h | 65 | File analysis and collection interface |
| Source/App/GlassLookAndFeel.cpp | 209 | Buttons, toggles, scrollbars, headers |
| Source/App/GlassLookAndFeel.h | 60 | Control styling interface |
| Source/App/GlassStyle.cpp | 244 | Backdrop, panels, pills |
| Source/App/GlassStyle.h | 102 | Design tokens and panel primitives |
| Source/App/LoudnessGraph.cpp | 442 | Plot, playhead, seeking |
| Source/App/LoudnessGraph.h | 73 | Graph interface |
| Source/App/Main.cpp | 92 | Application entry, window, look and feel |
| Source/App/MainComponent.cpp | 1019 | Layout, wiring, exports, transport |
| Source/App/MainComponent.h | 146 | Main view interface |
| Source/App/MediaFoundationAudioFormat.cpp | 518 | IMFSourceReader-backed reader |
| Source/App/MediaFoundationAudioFormat.h | 46 | AAC/M4A format interface |
| Source/App/PlaybackEngine.cpp | 189 | Device, transport, COM read-ahead thread |
| Source/App/PlaybackEngine.h | 68 | Transport interface |
| Source/App/TargetLibrary.cpp | 157 | targets.json parsing and user copy |
| Source/App/TargetLibrary.h | 48 | Target loading interface |
| Source/App/VerdictPanel.cpp | 158 | Verdict rows and fix hints |
| Source/App/VerdictPanel.h | 35 | Verdict list interface |
| Source/Engine/AnalysisResult.h | 52 | The result struct every layer consumes |
| Source/Engine/AudioAnalyser.cpp | 149 | Meter composition, mono downmix, erosion |
| Source/Engine/AudioAnalyser.h | 74 | Orchestrator interface, dialogue gating |
| Source/Engine/KWeighting.cpp | 81 | Per-sample-rate coefficient derivation |

| File | Lines | Purpose |
|---|---|---|
| Source/Engine/KWeighting.h | 69 | K-weighting filter design and biquad |
| Source/Engine/LoudnessMeter.cpp | 304 | Sub-block accumulation, gating, LRA |
| Source/Engine/LoudnessMeter.h | 107 | Loudness measurement interface and gating helpers |
| Source/Engine/QualityChecks.cpp | 188 | Peak, clipping, correlation |
| Source/Engine/QualityChecks.h | 88 | Sample-domain check interface |
| Source/Engine/TruePeakDetector.cpp | 278 | Polyphase oversampling and over events |
| Source/Engine/TruePeakDetector.h | 95 | True-peak interface and envelope type |
| Source/Report/JsonReport.cpp | 283 | Serialisation with null discipline |
| Source/Report/JsonReport.h | 45 | JSON writer interface |
| Source/Report/PdfReport.cpp | 510 | Page layout, graph, decimation |
| Source/Report/PdfReport.h | 47 | Report item and options |
| Source/Report/PdfWriter.cpp | 297 | PDF object model, xref, content streams |
| Source/Report/PdfWriter.h | 83 | Minimal PDF primitive interface |
| Source/Verdict/Target.h | 50 | Delivery specification struct |
| Source/Verdict/VerdictEngine.cpp | 288 | Pass/fail rules and fix hints |
| Source/Verdict/VerdictEngine.h | 69 | Status model and verdict types |
| Tools/MakeIcon.cpp | 177 | Application icon generator |
| Tools/UiSnapshot.cpp | 104 | Offscreen interface renderer |

### *Tests*

| File | Lines |
|---|---|
| Tests/AacTests.cpp | 275 |
| Tests/BatchTests.cpp | 250 |
| Tests/EngineTests.cpp | 499 |
| Tests/FileTests.cpp | 252 |
| Tests/JsonTests.cpp | 178 |
| Tests/PdfTests.cpp | 303 |
| Tests/PlaybackTests.cpp | 214 |
| Tests/SignalUtils.h | 91 |
| Tests/TestHarness.h | 101 |
| Tests/VerdictTests.cpp | 255 |
| Tests/main.cpp | 6 |

# 28. References


- ITU-R BS.1770-4 — Algorithms to measure audio programme loudness and true-peak audio level.
- EBU Tech 3341 — Loudness metering: EBU Mode metering to supplement loudness normalisation.
- EBU Tech 3342 — Loudness range: a measure to supplement loudness normalisation.
- EBU R 128 — Loudness normalisation and permitted maximum level of audio signals.
- ATSC A/85 — Techniques for establishing and maintaining audio loudness for digital television.